

StudyPlannerAI — Java + Spring Boot

Oct 2, 2026 · @Matheus e Gabriel

Aplicação REST para Planejamento de Estudos com IA

1. Objetivo

Desenvolver uma aplicação backend utilizando Java e Spring Boot para cadastrar as matérias e datas de prova de um estudante e, a partir delas, gerar um cronograma de estudos com inteligência artificial (API da OpenAI), por meio de uma API REST.

2. Requisitos funcionais

- **RF01 — Cadastrar matéria:** permitir o cadastro de uma nova matéria com sua data de prova.
- **RF02 — Consultar matéria:** permitir a consulta de uma matéria por seu identificador.
- **RF03 — Listar matérias:** permitir a consulta de todas as matérias cadastradas.
- **RF04 — Atualizar matéria:** permitir a alteração dos dados de uma matéria existente.
- **RF05 — Excluir matéria:** permitir a exclusão de uma matéria existente.
- **RF06 — Gerar cronograma:** gerar, com IA, um cronograma de estudos a partir das matérias cadastradas.
- **RF07 — Horas disponíveis:** permitir informar quantas horas por dia o estudante tem para estudar ao gerar o cronograma.

3. Dados da matéria

A versão 1 tem uma única entidade, **Matéria**, equivalente ao FoodItem do projeto original.

Cada matéria deve possuir, no mínimo, os seguintes dados:

Campo	Tipo sugerido	Descrição
id	Long	Identificador único da matéria.
nome	String	Nome da matéria.
area	Enum	Área do conhecimento: EXATAS, HUMANAS, BIOLOGICAS ou LINGUAGENS.
dificuldade	Enum	Dificuldade percebida: FACIL, MEDIA ou DIFICIL.

Campo	Tipo sugerido	Descrição
horasEstudadas	Integer	Horas já dedicadas à matéria.
dataProva	LocalDate	Data da próxima prova da matéria.

4. API REST

A aplicação deverá disponibilizar endpoints REST para as operações de CRUD e para a geração do cronograma. A estrutura abaixo é uma sugestão; os endpoints podem ser organizados de forma equivalente, desde que as operações sejam atendidas.

Método	Endpoint	Finalidade
POST	/materias	Cadastrar uma matéria.
GET	/materias/{id}	Consultar uma matéria pelo ID.
GET	/materias	Listar todas as matérias.
PUT	/materias/{id}	Atualizar uma matéria.
DELETE	/materias/{id}	Excluir uma matéria.
GET	/cronograma?horasPorDia={n}	Gerar o cronograma de estudos com IA (parâmetro opcional).

5. Exemplo de requisição

Exemplo de corpo para criação de uma matéria:

```
{
  "nome": "Cálculo I",
  "area": "EXATAS",
  "dificuldade": "DIFICIL",
  "horasEstudadas": 4,
  "dataProva": "2026-10-20"
}
```

Exemplo de geração do cronograma com 3 horas disponíveis por dia: GET /cronograma?horasPorDia=3

6. Requisitos técnicos

- Utilizar Java.
- Utilizar Spring Boot.
- Disponibilizar uma API REST.
- Utilizar persistência de dados em banco relacional de sua preferência.
- Utilizar LocalDate para datas e Enum para área e dificuldade.
- Integrar com a API da OpenAI de forma não bloqueante (WebClient).
- Ler a chave da API de variável de ambiente; ela nunca deve aparecer no código ou no repositório.
- Organizar o projeto separando responsabilidades entre camadas (controller, service, repository, model).
- Utilizar tratamento adequado dos códigos HTTP e dos erros da API.
- Disponibilizar instruções para execução do projeto.

7. Integração com IA

- A IA deve atuar como um professor e orientador de estudos, definido na mensagem de sistema.
- Devem ser enviadas à IA a data atual, as horas disponíveis por dia e a lista de matérias com prova futura, incluindo os dias restantes até cada prova.
- O cronograma deve priorizar provas mais próximas e matérias mais difíceis, não ultrapassar as horas disponíveis e reservar uma revisão na véspera de cada prova.
- A IA deve usar apenas as matérias fornecidas, sem inventar outras.
- A resposta deve seguir sempre a mesma estrutura: visão geral das prioridades, cronograma dia a dia até a próxima prova e dicas de método para cada matéria.
- Falhas na comunicação com a OpenAI devem resultar em resposta HTTP apropriada, sem expor detalhes internos.

8. Validações esperadas

- Nome da matéria deve ser informado.
- Data da prova deve ser informada e não pode estar no passado no momento do cadastro.
- Área e dificuldade devem ser valores válidos da lista definida.
- Horas estudadas devem ser válidas e não negativas.
- Horas por dia, quando informadas, devem estar entre 1 e 12.

- Uma matéria inexistente deve resultar em resposta HTTP apropriada.
- Operações de atualização e exclusão devem validar a existência do registro.
- Sem matérias com prova futura, a geração do cronograma não deve chamar a IA e deve retornar resposta apropriada.
- A API deve retornar respostas HTTP coerentes com cada operação.

9. Diferenciais

Os itens abaixo não são obrigatórios, mas podem ser utilizados para demonstrar conhecimentos adicionais:

- Testes unitários e/ou de integração, simulando a resposta da IA.
- Documentação da API com OpenAPI/Swagger.
- Docker/Docker Compose.
- Uso de migrations para o banco de dados.
- Resposta da IA em JSON, mapeada para um objeto estruturado.
- Histórico dos cronogramas gerados, salvo em banco.
- Filtros por área ou por provas nos próximos dias.
- Paginação na listagem de matérias.
- Validações utilizando Bean Validation.
- Tratamento global de exceções.
- Logs e organização adequada das configurações da aplicação.

10. Entrega

Entregar o código-fonte do projeto em um repositório Git, contendo um README com as instruções necessárias para configurar, executar e testar a aplicação.

O README deve informar também quais tecnologias foram utilizadas, os parâmetros de configuração do banco de dados e como configurar a variável de ambiente com a chave da OpenAI.

11. Critérios de avaliação

Critério	O que será observado
Funcionamento	CRUD completo, geração do cronograma e API funcionando corretamente.

Critério	O que será observado
Código	Clareza, organização, legibilidade e boas práticas.
Spring Boot	Uso adequado dos recursos do framework.
Persistência	Modelagem e acesso aos dados.
API REST	Endpoints, métodos HTTP, status codes e contratos.
Integração com IA	Prompt bem estruturado, coerência do cronograma e tratamento de falhas da API externa.
Validações	Tratamento de entradas inválidas e cenários de erro.
Testes	Cobertura e qualidade dos testes, quando apresentados.
Documentação	Facilidade para executar e compreender o projeto.

Observação: é possível tomar decisões de implementação diferentes das sugestões deste documento, desde que os requisitos principais sejam atendidos.